BCA Semester 1 Examination 2025

Model Solutions: 1BCA4 - Programming in C

Session	Paper Code	Time	Full Marks	Pass Marks
2025-2028	111604	3 Hours	100	40

Instructions: Answer any five questions (equal marks). Solutions for all ten questions are given below for practice. All programs were compiled with GCC. Write answers in your own words in the exam.

Q1. Discuss different searching techniques in array. Explain with an example.

Searching means finding the position of a given value (key) in an array. The two basic techniques are linear search and binary search.

1. Linear (sequential) search

Each element is compared with the key one by one from the first element until the key is found or the array ends. It works on unsorted or sorted data. Time complexity: best O(1), worst/average O(n).

```
#include <stdio.h>
int main()
{
    int a[50], n, i, key, found = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter element to search: ");
    scanf("%d", &key);
    for (i = 0; i < n; i++)
    {
        if (a[i] == key)
        {
            printf("Found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("Element not found\n");
    return 0;
}
```

Example: array {10, 25, 7, 40}, key = 7: compare 10, 25, then 7 matches, so it is found at position 3.

2. Binary search

Works only on a **sorted** array. The key is compared with the middle element; if equal, the search ends; if smaller, search continues in the left half, otherwise in the right half. The search range is halved each time. Time complexity: O(log n).

```
#include <stdio.h>
int main()
{
    int a[50], n, i, key, low, high, mid, found = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements in ascending order: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    printf("Enter element to search: ");
    scanf("%d", &key);
    low = 0;
    high = n - 1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (a[mid] == key)
        {
            printf("Found at position %d\n", mid + 1);
            found = 1;
            break;
        }
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    if (!found)
        printf("Element not found\n");
    return 0;
}
```

Example: array {5, 12, 19, 26, 33}, key = 26: mid = 19 (26 is greater, go right), then low = 3, high = 4, mid = 26, found at position 4.

Basis	Linear search	Binary search
Array condition	Sorted or unsorted	Must be sorted
Complexity	O(n)	O(log n)
Speed	Slow for large data	Very fast
Logic	Simple	Slightly complex

Q2. Write a C program to count and print the number of vowels in a given string.

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    char str[100];
    int i, count = 0;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    for (i = 0; str[i] != '\0'; i++)
    {
        switch (tolower(str[i]))
        {
            case 'a': case 'e': case 'i': case 'o': case 'u':
                count++;
                printf("%c ", str[i]);
        }
    }
    printf("\nNumber of vowels = %d\n", count);
    return 0;
}
```

Sample output

```
Enter a string: Programming in C
o a i i
Number of vowels = 4
```

The loop runs until the null character '\0'. tolower() makes the check case-insensitive, so both 'A' and 'a' are counted.

Q3. Write short notes on any two of the following: (i) Static storage class (ii) Preprocessor directive (iii) sizeof operator

(i) Static storage class

The **static** storage class creates a variable that keeps its value between function calls. A static local variable is created once, initialised only once (default value 0), stored in the data segment and lives for the whole program, but is visible only inside the function/block. A static global variable/function is visible only within its own source file.

```
#include <stdio.h>
void counter()
{
    static int c = 0;   /* initialised only once */
    c++;
    printf("Called %d time(s)\n", c);
}
int main()
{
    counter();   /* Called 1 time(s) */
    counter();   /* Called 2 time(s) */
    counter();   /* Called 3 time(s) */
    return 0;
}
```

(ii) Preprocessor directive

Preprocessor directives are instructions beginning with # that are processed by the preprocessor before compilation. They need no semicolon. Main types:

- **File inclusion:** #include <stdio.h>
- **Macro definition:** #define, #undef (constants and function-like macros)
- **Conditional compilation:** #if, #ifdef, #ifndef, #else, #endif
- **Others:** #pragma, #error

```
#include <stdio.h>
#define PI 3.14159
#define SQUARE(x) ((x) * (x))
#define DEBUG
int main()
{
    float r = 2;
    printf("Area = %f\n", PI * SQUARE(r));
#ifdef DEBUG
    printf("Debug mode on\n");
#endif
    return 0;
}
```

(iii) sizeof operator

sizeof is a compile-time unary operator that returns the size, in bytes, of a data type, variable or array. The result type is size_t (print with %zu). Example: sizeof(int) is usually 4, sizeof(double) 8, and for int arr[10] it is 40.

```
#include <stdio.h>
int main()
{
    int a;
```

```
    double d;
    int arr[10];
    printf("%zu %zu %zu %zu\n", sizeof(char), sizeof(a), sizeof(d), sizeof(arr));
    /* typical output: 1 4 8 40 */
    return 0;
}
```

Q4. Explain user defined function and types of user defined function. Write a program to find factorial of a number using argument with return.

A user defined function is a named block of code written by the programmer to perform a specific task. It makes the program modular, avoids repetition, and is easy to test and reuse. A function has three parts: **declaration (prototype)**, **definition** and **call**.

```
return_type function_name(parameter list)
{
    /* body */
    return value;
}
```

Types of user defined function

Type	Example
1. No arguments, no return value	void hello(void)
2. Arguments, no return value	void show(int a)
3. No arguments, with return value	int getNumber(void)
4. Arguments with return value	int add(int a, int b)

Program: factorial using function with argument and return value

```
#include <stdio.h>
long int factorial(int n);          /* function prototype */
int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("Factorial of %d = %ld\n", num, factorial(num));
    return 0;
}
long int factorial(int n)           /* function with argument and return value */
{
    long int f = 1;
    int i;
    for (i = 1; i <= n; i++)
        f = f * i;
    return f;
}
```

```
Enter a number: 5
Factorial of 5 = 120
```

Q5. Write programs for the following: (i) Find all palindrome numbers from 1 to 1000 (ii) Print table and sum of table of any inputted number

(i) Palindrome numbers from 1 to 1000

A palindrome reads the same from both sides (e.g. 121). The digits of each number are reversed and compared with the original.

```
#include <stdio.h>
int main()
{
    int n, temp, rev, rem;
    printf("Palindrome numbers from 1 to 1000:\n");
    for (n = 1; n <= 1000; n++)
    {
        temp = n;
        rev = 0;
        while (temp > 0)
        {
            rem = temp % 10;
            rev = rev * 10 + rem;
            temp = temp / 10;
        }
        if (rev == n)
            printf("%d ", n);
    }
    printf("\n");
    return 0;
}
```

Output begins: 1 2 3 4 5 6 7 8 9 11 22 33 ... 99 101 111 121 ... 999 (all single-digit numbers, two-digit numbers like 11 to 99 with equal digits, and three-digit numbers like 101, 111, 121).

(ii) Table and sum of table

```
#include <stdio.h>
int main()
{
    int n, i, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= 10; i++)
    {
        printf("%d x %d = %d\n", n, i, n * i);
        sum = sum + n * i;
    }
    printf("Sum of the table = %d\n", sum);
    return 0;
}
```

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
Sum of the table = 275
```

Q6. Explain the switch statement in detail with a suitable example.

The switch statement is a multi-way decision statement. It compares the value of an expression with a list of constant case labels and executes the matching block. It is a cleaner alternative to a long if-else-if ladder.

```
switch (expression)
{
    case constant1: statements; break;
    case constant2: statements; break;
    ...
    default: statements;
}
```

Rules

- The expression must give an integer or character value (not float).
- Case labels must be unique constants; duplicate cases are not allowed.
- **break** ends the switch; without it execution falls through into the following cases.
- **default** is optional, runs when no case matches, and may be placed anywhere (usually last).
- Relational/range conditions (e.g. x > 10) cannot be used in case labels.

Example: simple calculator

```
#include <stdio.h>
int main()
{
    float a, b;
    char op;
    printf("Enter expression (e.g. 5 + 3): ");
    scanf("%f %c %f", &a, &op, &b);
    switch (op)
    {
        case '+': printf("Result = %.2f\n", a + b); break;
        case '-': printf("Result = %.2f\n", a - b); break;
        case '*': printf("Result = %.2f\n", a * b); break;
        case '/':
            if (b != 0)
                printf("Result = %.2f\n", a / b);
            else
                printf("Division by zero!\n");
            break;
        default:
            printf("Invalid operator\n");
    }
    return 0;
}
```

Q7. What do you mean by String? Discuss four library functions related to string with example.

A string is a sequence of characters stored in a character array and terminated by the null character **'\0'**. Example: `char name[10] = "Ravi";` stores R a v i \0. Strings are read with scanf("%s") / gets / fgets and printed with printf("%s") / puts. String library functions are declared in **<string.h>**.

Function	Purpose	Example
strlen(s)	Returns length of string (without \0)	strlen("India") = 5
strcpy(d, s)	Copies string s into d	strcpy(d, "abc") makes d = "abc"

Function	Purpose	Example
strcat(d, s)	Appends s at the end of d	"Good" + "Day" gives "GoodDay"
strcmp(s1, s2)	Compares two strings: 0 if equal, negative if s1 < s2, positive if s1 > s2	strcmp("abc","abc") = 0

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[50] = "Hello", s2[50] = "World", s3[50];
    printf("Length of s1 = %zu\n", strlen(s1));      /* 5 */
    strcpy(s3, s1);                                  /* s3 = "Hello" */
    printf("Copied string = %s\n", s3);
    strcat(s1, s2);                                  /* s1 = "HelloWorld" */
    printf("Joined string = %s\n", s1);
    if (strcmp(s2, s3) == 0)
        printf("Strings are equal\n");
    else
        printf("Strings are not equal\n");
    return 0;
}
```

Other useful functions: strrev (non-standard), strlwr/strupr (non-standard), strncpy, strncmp, strchr, strstr.

Q8. What is an operator? Discuss relational and logical operators with example. Also discuss unary and binary operators.

An operator is a symbol that tells the compiler to perform a specific mathematical, relational or logical operation on operands. Categories: arithmetic, relational, logical, assignment, increment/decrement, bitwise, conditional (?:) and special (sizeof, comma).

Relational operators

Compare two values and give 1 (true) or 0 (false).

Operator	Meaning	Example (a=10, b=20)	Result
<	Less than	a < b	1
>	Greater than	a > b	0
<=	Less than or equal	a <= 10	1
>=	Greater than or equal	a >= b	0
==	Equal to	a == b	0
!=	Not equal to	a != b	1

Logical operators

Combine conditions and give 1 or 0.

Operator	Meaning	Example	Result
&&	Logical AND (true if both true)	(a < b) && (b > 15)	1
||	Logical OR (true if any is true)	(a > b) || (b > 15)	1
!	Logical NOT (reverses result)	!(a < b)	0

Unary and binary operators

- **Unary operators** work on a single operand: ++, --, unary minus (-a), !, ~, sizeof, & (address), * (pointer).
- **Binary operators** need two operands: + - * / %, relational, logical (&&, ||), assignment (=), bitwise (&, |, ^).
- There is also one **ternary** operator: ?: (e.g. max = (a > b) ? a : b;).

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    printf("%d %d %d\n", a < b, a == b, a != b);          /* 1 0 1 */
    printf("%d %d %d\n", (a < b) && (b > 15),             /* 1 */
                         (a > b) || (b > 15),             /* 1 */
                         !(a < b));                       /* 0 */
    printf("%d\n", -a);                                   /* unary minus: -10 */
    a++;                                                  /* unary increment: a = 11 */
    printf("%d\n", a + b);                                /* binary plus: 31 */
    return 0;
}
```

Q9. Write short notes on: (i) '\0' character (ii) identifier (iii) Escape sequence character (iv) Operator precedence

(i) '\0' (null character)

The null character has ASCII value 0. It marks the end of a string in C, so string functions know where the string stops. The string "HI" occupies 3 bytes: 'H', 'I', '\0'. It is not the same as the character '0' (ASCII 48).

(ii) Identifier

An identifier is the name given to a variable, function, array, structure etc. **Rules:** it may contain letters, digits and underscore; it must start with a letter or underscore; no spaces or special symbols; keywords (int, for, while) cannot be used; C is case sensitive (Total and total are different). Examples: `sum, _count, student_name, marks1`; invalid: `1abc, my-name, float`.

(iii) Escape sequence characters

Backslash character combinations used in strings to represent special (non-printable) characters.

Sequence	Meaning	Sequence	Meaning
\n	New line	\\	Backslash
\t	Horizontal tab	\"	Double quote
\a	Alert (beep)	\'	Single quote
\b	Backspace	\r	Carriage return
\0	Null character	\f	Form feed

(iv) Operator precedence

Precedence decides which operator is evaluated first in an expression; associativity decides the order for operators of equal precedence. Parentheses can override it.

Priority (high to low)	Operators	Associativity
1	() [] . ->	Left to right
2	! ~ ++ -- unary + - * & sizeof	Right to left

Priority (high to low)	Operators	Associativity
3	* / %	Left to right
4	+ -	Left to right
5	< <= > >=	Left to right
6	== !=	Left to right
7	&&	Left to right
8	||	Left to right
9	?:	Right to left
10	= += -= *= /= %=	Right to left
11	,	Left to right

Example: 2 + 3 * 4 = 14 (multiplication first), but (2 + 3) * 4 = 20.

Q10. How are break and continue statements used in C? Explain with loop example.

break

The break statement immediately terminates the nearest enclosing loop (for, while, do-while) or switch, and control passes to the statement after it. It is generally used with an if condition to leave a loop early (for example when a search key is found).

continue

The continue statement skips the remaining statements of the current iteration and moves to the next iteration of the loop. In a for loop control goes to the increment step; in while/do-while it goes to the condition test. The loop is not terminated.

```
#include <stdio.h>
int main()
{
    int i;
    printf("break example: ");
    for (i = 1; i <= 10; i++)
    {
        if (i == 6)
            break;            /* loop ends when i becomes 6 */
        printf("%d ", i);
    }
    printf("\ncontinue example: ");
    for (i = 1; i <= 10; i++)
    {
        if (i % 2 == 0)
            continue;         /* skip even numbers */
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

```
break example: 1 2 3 4 5
continue example: 1 3 5 7 9
```

break	continue
Terminates the loop completely.	Skips only the current iteration.

break	continue
Can be used in loops and switch.	Can be used only in loops.
Control goes outside the loop.	Control goes to the next iteration.